Operationalizing Human-AI Collaboration for High-Volume Project Risk Management

Marc Bara[1,2][0009-0005-1480-5760]

[1] ICSO Analytics, Barcelona, Spain
[2] ESADE Business School, Barcelona, Spain
marcoantonio.bara@esade.edu

Abstract. Artificial intelligence tools can now generate hundreds of potential risk scenarios per project, creating an "intelligence paradox" where AI's analytical power overwhelms rather than assists project managers. In previous work [1], we identified four human-AI collaboration patterns from cybersecurity, security screening, financial systems, and robotics that address high-volume risk management, and proposed a theoretical transfer framework to project management. However, that framework remained at the conceptual level, leaving a critical gap between theory and implementation. This paper operationalizes the framework into SHARP (Semantic Human-AI Risk Prioritization), defining: (a) a semantic triage system where AI classifies risks through contextual understanding rather than traditional probability-impact matrices, (b) a three-tier collaboration model (MONITOR/FLAG/ESCALATE) with clearly delineated AI and human responsibilities, and (c) a four-layer safety architecture ensuring the system fails safely when—not if—classification errors occur. We present an initial simulation-based evaluation using a synthetic project with planted ground-truth risks, processed through an LLM-based orchestration pipeline. The system is designed so that the project manager defines semantic classification rules while AI executes them at volume—preserving human judgment where it matters while delegating volume processing to AI.

Keywords: project risk management · human-AI collaboration · decision support systems · semantic risk triage · LLM orchestration · safe failure · human-AI teaming

1 Introduction

Project risk management faces a paradox. As AI tools become capable of analyzing project plans, communications, and historical data to identify potential risks, they can generate hundreds of risk scenarios for a single project—far exceeding any project manager's capacity to evaluate manually. We term this the *intelligence paradox*: AI's analytical power, naively applied, creates more information overload than it solves [1]. This challenge sits at the core of human-AI collaboration for real-world decision-making: how to leverage AI's processing capacity without sacrificing the contextual judgment that makes risk management effective.

This is not a hypothetical concern. Our prior systematic review [1] examined 344 papers across two search streams: 296 on AI-related project management challenges and 48 on human-AI collaboration frameworks in adjacent high-volume domains. Of the 296 PM-focused papers, 17 already document information overload challenges [3, 4, 10, 11], yet none propose systematic solutions adapted from domains that have already solved analogous problems.

This paper builds on two complementary prior lines of work. [1] (the "Catania" framework) identified four human-AI collaboration patterns from cybersecurity, security screening, financial systems, and robotics, mapped them to PM challenges through a transfer matrix, articulated three adaptation principles (context preservation, threshold calibration, integration requirements), and proposed a three-tier threshold system (MONITOR/FLAG/ESCALATE)—but at the conceptual level, leaving the operational design unspecified. In parallel, [2] demonstrated that an LLM-based pipeline can detect semantic deficiencies in risk register entries and reformulate them at scale, but applied a single uniform treatment to every risk: it had no triage, no tier-based delegation, no clustering, no longitudinal reclassification, and no safety architecture against misclassification.

SHARP (Semantic Human-AI Risk Prioritization) operationalizes Catania's framework while extending the LLM pipeline of [2] into a triage-aware system. Our contributions are threefold. First, we define a **semantic triage system** where risk classification is based on contextual understanding rather than numerical probability-impact scoring. Second, we specify the **precise responsibilities** of AI and human at each tier, including how the project manager configures the system through semantic rules and how the AI executes those rules at volume. Third, we develop a **four-layer safety architecture** that ensures the system fails safely when classification errors occur. We also present an initial evaluation on a synthetic project with planted ground-truth criticals.

2 Background: From Cross-Domain Patterns to SHARP

Our previous systematic review [1] established that successful human-AI collaboration in high-volume risk contexts follows distinct patterns. Four were identified and mapped to project management challenges through a transfer matrix (Table 1).

The framework also proposed three adaptation principles: *context preservation* (project risks are context-dependent, unlike universal cybersecurity threats), *threshold calibration* (PM decisions often involve irreversible commitments, requiring conservative thresholds), and *integration requirements* (the fragmented PM toolset landscape). A three-tier system was proposed: AI-managed monitoring for low-severity risks, AI-flagged human review for moderate risks, and direct human intervention for high-severity risks. This paper operationalizes that system into SHARP.

Table 1. Collaboration patterns identified in [1] and their PM adaptation

Pattern	**Source Domain**	**Core Logic**	**PM Adaptation**
Alert Prioritization [5]	Cybersecurity	AI clusters thousands of alerts to surface attack patterns	AI identifies risk clusters and dangerous combinations
Dual-Strategy Triage [6]	Security Screening	Clear/flag/escalate classification of items	AI auto-clears low risks, escalates those needing human judgment
Weak-Signal Detection [7]	Financial Systems	Monitor data streams for early warning signs	AI scans project communications for emerging risk indicators
Multi-Unit Coordination [8]	Robotics	Aggregate status data across units, suggest reallocation	AI consolidates cross-team risks, identifies conflicts

3 Operationalizing SHARP

The operationalization of SHARP addresses three questions left open by the theoretical framework: how classification decisions are made, what each actor does at each tier, and how the system adapts as the project evolves.

3.1 Semantic Rules: The Human Configures, the AI Executes

A fundamental design decision distinguishes this approach from traditional risk matrices [9]: classification is based on **semantic rules defined by the project manager**, not on numerical probability-impact scores.

At project initiation, the PM configures the system with rules expressed in natural language:

– "Anything affecting the critical path within 4 weeks → minimum FLAG"
– "More than one risk linked to the same supplier → FLAG"
– "Any risk affecting client relationship → ESCALATE"
– "If the AI has low confidence in its assessment → FLAG, never MONITOR"
– "During weeks 8–12 (critical phase) → lower all thresholds"

These rules encode the PM's contextual knowledge—their understanding of stakeholders, organizational dynamics, and strategic priorities. The AI cannot generate these rules; this is the irreplaceable human contribution. However, once defined, a large language model (LLM) can interpret and apply them at scale across hundreds of risks simultaneously.

This design directly implements the *context preservation* principle from [1]: the human retains strategic judgment by designing the policy; the AI handles volume execution.

3.2 AI Semantic Reading

For each risk, the AI performs a contextual semantic analysis rather than computing a numerical score. The LLM evaluates: (1) the risk's semantic meaning and affected project components, (2) connections with other risks that could create amplification, (3) critical path proximity and timing margins, (4) reversibility of impact if the risk materializes, (5) corroborating signals in project communications, and (6) the AI's own confidence in its assessment—where low confidence is itself a reason to escalate.

This semantic reading, combined with the PM's rules, determines tier classification. The distinction from traditional approaches is significant: a probability-impact matrix treats risks as independent data points with numerical coordinates, while semantic reading treats each risk as a contextualized narrative within the broader project story.

Confidence is self-reported in the structured output. The Analyst's prompt constrains the response to a JSON schema in which every classification carries a `confidence` field on $[0, 1]$ alongside its tier and justification. The model is instructed to interpret this field as its self-assessed confidence in the tier choice given the semantic content and the PM rules, and the system prompt explicitly states that "if AI confidence is below 0.60, classify FLAG, never MONITOR" (the threshold is parametrized in code and documented in the reproducibility section). This makes low confidence a first-class architectural trigger rather than a post-hoc inspection: the safety rule operates inside the classification call. We chose self-reported confidence over alternatives such as log-probability calibration or a secondary verifier because it composes cleanly with structured output and is interpretable in the same prose vocabulary the PM uses to write rules. The confidence ordering observed in the static pass (Section 4.4) is consistent with the intended diagnostic role of confidence, though calibration and run-to-run stability remain to be evaluated.

3.3 Three-Tier Collaboration Model

Tier 1—MONITOR (AI decides, PM defines rules). When the semantic reading indicates low impact, no critical path connection, no match with PM escalation rules, high reversibility, and high AI confidence, the system automatically assigns "accept and monitor." This is the only tier where the AI makes the response decision autonomously. Crucially, the AI continuously re-evaluates these risks with every new project event, reclassifying upward if context changes. Expected volume: approximately 65–70%. The rationale mirrors cybersecurity operations, where analysts configure rules for auto-closure of low-priority alerts rather than reviewing each one individually [5].

Tier 2—FLAG (AI proposes, PM decides). Triggered by moderate impact, high uncertainty, a match with PM escalation rules, low AI confidence, or cluster membership. The AI proposes a concrete response with semantic justification—e.g., "mitigate by renegotiating the timeline with Supplier X, because this risk amplifies Risk #47 and both converge in Week 12." The PM

reviews, applies contextual knowledge (political relationships, contractual constraints, sponsor tolerance), and accepts, modifies, or rejects. Expected volume: approximately 20–25%.

Tier 3—ESCALATE (AI informs, PM acts with team). Reserved for high impact on project objectives, low reversibility, systemic threats from multiple connected risks, or PM direct escalation rules (client relationship, safety, compliance). The AI prepares a complete dossier but does *not* decide the response. The PM works on these with their team, stakeholders, and sponsors. Expected volume: approximately 5–10%.

Table 2 summarizes the collaboration model.

Table 2. Three-tier collaboration model: AI and human responsibilities

Tier	Volume	AI Responsibility	PM Responsibility	Risk Response
MONITOR	65–70%	Apply PM rules, classify, monitor, reclassify if context changes	None (unless promoted)	AI decides: accept + monitor
FLAG	20–25%	Propose response with semantic justification	Review, add context, decide	PM decides
ESCALATE	5–10%	Prepare complete dossier	Act with team, stakeholders	PM designs with team

3.4 Dynamic Reclassification

Risks are not static. Every new project event triggers a reclassification cycle across the affected portion of the register (the exact scope is specified below). A MONITOR risk may be promoted to FLAG when clustering detects a new connection with another risk, or when a communication signal confirms a concern. A FLAG risk may be promoted to ESCALATE when multiple related risks converge. Conversely, an ESCALATE risk may be demoted to FLAG after the PM's response reduces the threat, and a FLAG risk may return to MONITOR if the underlying condition is resolved.

Additionally, the PM's phase-based rules activate dynamically. When the project enters a pre-defined critical phase, thresholds drop globally: risks that were classified as MONITOR may be automatically promoted to FLAG. This is not the AI deciding to become more cautious—it is the PM's pre-configured policy activating on schedule.

Computational scope of a reclassification cycle. For each project event, the orchestrator does not blindly re-evaluate every risk in the register. It computes an *affected set* by intersecting the event with structural filters: risks sharing the same supplier, team, or critical-path segment as the event; existing

FLAG/ESCALATE risks whose clusters could grow; and a rolling window of recent MONITOR risks that may have crossed a threshold. Only risks in the affected set are re-sent to the Analyst, batched to respect context-window limits. This bounds per-event cost from $O(N)$ to roughly $O(k)$ where k is the size of the affected set, typically a small fraction of the register in large projects. A full periodic re-evaluation can still be scheduled (e.g., weekly) as a safety net, but is not the per-event path. The minimal pipeline reported in Section 4.4 exercises only the initial static classification; per-event filtered reclassification is left to follow-on work.

4 Simulation-Based Evaluation Design

To test whether SHARP produces a manageable, high-quality risk picture from an AI-generated risk explosion, we use a synthetic project with controlled ground truth. We first describe the full evaluation design and then narrow to the static slice exercised in this paper.

4.1 Synthetic Project with Planted Ground Truth

Full experimental design (envisioned). The full design envisions a synthetic project with multiple teams, vendors, and critical-path dependencies; a complete plan including scope, schedule, WBS, organizational structure, budget, and an initial 20–30-risk register; and a timeline of 15–20 simulated events representing triggers that in production would arrive continuously—e.g., a supplier confirming a 5-day delay in Week 3, a technician emailing "not sure we'll make the integration deadline" in Week 4, a client-driven scope change in Week 6, and the project entering its critical-path phase in Week 13. Within this scenario, ground-truth "mines" would be planted: critical risks, dangerous combinations invisible as individual entries but lethal together, weak signals buried in simulated communications, and interface gaps no single team owns. These would constitute the ground truth against which the full SHARP loop is evaluated end-to-end; the framework never has access to them and must discover them through its own analysis.

Static slice exercised in this paper. The evaluation reported here uses a minimal slice of that design: an 80-risk register with planted tier-level ground truth (12 ESCALATE, 18 FLAG, 50 MONITOR), processed in a single static classification pass without streamed events and without AI-generated risk amplification. The aim is to characterize the initial triage decision and the conservative-escalation behavior of the safety architecture before the longitudinal mechanisms (event-driven reclassification, clustering, weak-signal scanning) are added on top. The dataset and the metrics that follow refer to this static slice; the multi-event extension is framed as follow-on work in Section 4.4.

4.2 System Architecture

The architecture below describes the intended full SHARP system. The evaluation reported in Section 4.4 implements only the static triage subset (the SQLite store, the Analyst's initial classification path, and the orchestrator wiring); the Generator, the clustering and weak-signal stages of the Analyst, the event-driven reclassification loop, and the Streamlit dashboard are not exercised in the present evaluation. The full system consists of five components (Fig. 1): (1) a **SQLite database** as single source of truth for all risks, classifications, justifications, and reclassification history; (2) an **LLM Generator** that receives project inputs and produces risks in deliberately unfiltered batches—targeting approximately 500 from initial plan analysis alone; (3) an **LLM Analyst** that applies the triage framework in batches of 30–50 risks (respecting context window constraints), performs clustering on FLAG/ESCALATE risks, scans communications for weak signals, and reclassifies existing MONITOR risks; (4) a **Python orchestrator** managing the pipeline sequence; and (5) a **Streamlit dashboard** for visualization.

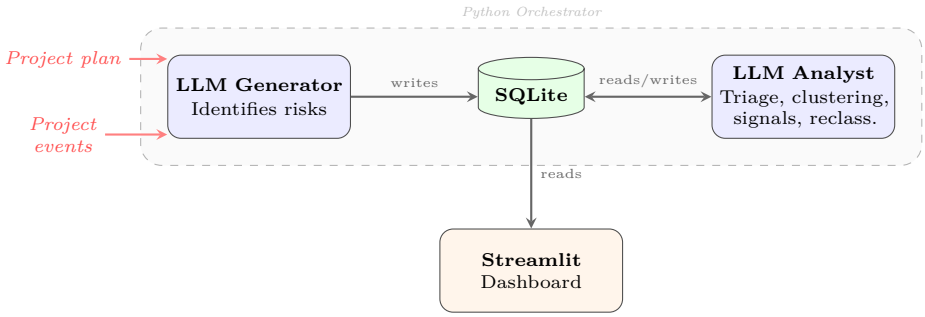

Fig. 1. System architecture. The LLM Generator identifies risks from the project plan and subsequent events; the LLM Analyst reads risks from SQLite, applies triage, clustering, and signal scanning, and writes classifications back. A Python orchestrator (dashed) coordinates the pipeline. The dashboard reads from SQLite for visualization.

For each simulated event, the orchestrator executes: (i) feed the event to the Generator for new risks, (ii) triage new risks, (iii) reclassify existing MONITOR risks against new context, (iv) cluster all FLAG/ESCALATE risks, (v) scan communications for weak signals, and (vi) update the database.

Reproducibility. The LLM pipeline inherits its plumbing from [2]: the seven-field risk schema (Risk ID, Type, Root Cause, Incident, Impact, Response Strategy, Response Action), batch processing, and structured-output handling. SHARP extends this base in four ways: (a) replacing the single-task quality-enhancement prompt with a triage system prompt encoding the PM's semantic rules and tier definitions, (b) constraining the output via JSON-schema structured outputs to return `tier`, `confidence`, and `justification` per risk, (c) substituting SQLite

for the CSV round-trip to support longitudinal analysis, and (d) adding ground-truth labels for evaluation. The Analyst runs on Claude Sonnet 4.6 with adaptive thinking disabled, effort set to `low`, and prompt caching enabled on the system block—the system prompt is identical across batches, so batches 2–N read from cache. The classification batch size is 20. The full system prompt, dataset, SQLite schema, and metrics code accompany the camera-ready version.

Database schema. The SQLite store uses three tables. The `risks` table holds one row per risk with the seven semantic fields (type, root cause, incident, impact, response strategy, response action, plus an optional `ground_truth_tier` used only for evaluation). The `classifications` table is append-only: each tier assignment generates a new row keyed by `risk_id` with `tier`, `confidence`, `justification`, model identifier, timestamp, and optional `triggering_event_id`. This append-only design preserves the full reclassification history of each risk, enabling the longitudinal analysis required to study tier drift over the project lifecycle. The `events` table records project events that trigger reclassification cycles, with a foreign-key relationship into `classifications`.

4.3 Evaluation Metrics

Metrics enabled by the full design. The controlled ground truth enables precise measurement of *critical risk retention rate* (planted critical risks correctly classified as FLAG/ESCALATE), *volume reduction* (percentage classified as MONITOR), *cluster detection accuracy* (identification of planted dangerous combinations), *initial false negative rate* (critical risks wrongly sent to MONITOR on first pass), *recovery rate* (false negatives correctly reclassified after subsequent events), and *time to recovery* (events required for self-correction).

Metrics reported for the static slice. The static pass exercised in this paper reports the subset that does not require streamed events: critical retention rate, initial false-negative rate, volume reduction, per-tier recall, exact-match accuracy, mean confidence by assigned tier, and end-to-end API cost. Cluster detection accuracy, recovery rate, and time to recovery require the multi-event loop and are deferred to follow-on work.

4.4 Results

We report initial results from a minimal end-to-end execution of the pipeline: a single static classification pass over a synthetic project of 80 risks with planted ground truth (12 ESCALATE, 18 FLAG, 50 MONITOR). The full multi-event simulation with dynamic reclassification, clustering, and weak-signal scanning is deferred to follow-on work. The static pass exercises the initial triage mechanism and the conservative-escalation component of the safety architecture (Layer 1); the longitudinal safety mechanisms—event-driven reclassification (Layer 2), PM override enforcement under conflicting signals (Layer 3), and dashboard-mediated audit (Layer 4)—remain to be tested in the full multi-event simulation.

Headline metrics. The system achieved a **critical retention rate of 100%** (12/12 planted ESCALATE risks placed in FLAG or ESCALATE) and an **initial**

false-negative rate of 0%—no critical risk was silently routed to MONITOR. The **volume reduction to MONITOR was 66.2%** (53/80 risks), within the 65–70% range anticipated by the framework. Per-tier exact-match recall was MONITOR 50/50 = 100%, FLAG 11/18 = 61.1%, ESCALATE 12/12 = 100%; overall exact-match accuracy was 73/80 = 91.2%. Table 3 reports the full confusion matrix.

Table 3. Confusion matrix on the static classification pass (rows = planted ground truth, columns = AI classification)

GT \ AI	MONITOR	FLAG	ESCALATE
MONITOR	50	0	0
FLAG	3	11	4
ESCALATE	0	0	12

Evidence of conservative bias (Layer 1). Every classification error on the FLAG row went *upward* (4 to ESCALATE) or to MONITOR for low-impact reversible items (3); no error crossed the boundary that the safety architecture is designed to prevent (a critical risk leaking into MONITOR). The 4 FLAG-to-ESCALATE over-escalations all involved triggers the PM rules explicitly flag for escalation: a production-incident-likely configuration drift, an SLA breach on enterprise contracts, vendor concentration across multiple risks, and a single point of failure in on-call coverage. These are defensible escalations rather than spurious noise, and they are consistent with the system being asymmetrically tuned to favor false positives over false negatives (Layer 1 of the safety architecture, Section 5).

Confidence as a diagnostic signal. Mean AI-reported confidence ordered as expected by design: ESCALATE 0.95, MONITOR 0.79, FLAG 0.75. The FLAG tier carrying the lowest mean confidence is consistent with the intended role of FLAG as the ambiguous middle band, and suggests that self-reported confidence may be useful as a trigger for human review (Layer 1, Section 5). Calibration and run-to-run stability of this signal remain to be evaluated.

Cost and baseline. The full 80-risk classification ran in four batches of 20 against Claude Sonnet 4.6 with prompt caching on the system block; total cost was approximately US$0.07 (batches 2–4 read the system prompt from cache at ∼10% of base input price). For comparison, the LLM-only baseline from [2]—which uses the same model family on the same risk schema but without the triage architecture—returns a single undifferentiated list with no tier signal, no confidence calibration, and no safety-driven prioritization, leaving the PM with the same 80-risk review load that motivated this work. SHARP reduces that load to the 27 risks (FLAG + ESCALATE) the PM actively needs to attend to, while guaranteeing that the 12 planted criticals are inside that 27.

5 Discussion: Reliability and Safe Failure

The question "LLMs are unreliable—how can you trust risk classification to one?" requires direct engagement. Our answer is not that the system is infallible, but that it is designed to **fail safely**, and that its failure mode is superior to current practice.

The only dangerous error is classifying a critical risk as MONITOR—because then no human examines it. Other errors are tolerable: a trivial risk classified as FLAG wastes minutes of PM time; a moderate risk classified as ESCALATE receives excess attention. The safety architecture specifically targets the dangerous error through four layers.

Layer 1: Conservative bias. The LLM Analyst is configured to classify upward when uncertain. Low confidence mandates FLAG, never MONITOR. This produces more false positives but fewer false negatives—the same asymmetry used in medical screening. **Layer 2: Continuous reclassification.** A wrongly classified MONITOR risk is re-evaluated with every project event; if genuinely critical, subsequent signals will promote it. **Layer 3: PM override rules.** For non-negotiable domains (safety, compliance, client relations), the PM blocks MONITOR classification entirely—the human sets the floor. **Layer 4: Transparency.** Every classification includes its justification; the PM can query and manually promote any MONITOR risk. The dashboard operationalizes this without overwhelming the PM by inverting the default visibility: the FLAG and ESCALATE queues are shown expanded with full justifications and confidence scores, while the MONITOR queue is collapsed to a count with on-demand drill-down. A PM who wants to audit MONITOR decisions can filter by confidence (showing low-confidence MONITOR risks first, where review is most warranted), by escalation-rule keyword, or by free-text search over the justifications. Manual override is a single click that creates a new row in the append-only `classifications` table, preserving the audit trail. The design principle is that transparency must be *available* rather than *forced*: the PM sees what needs attention by default, and can excavate the rest only when a specific question arises.

Limitations. Synthetic project data cannot fully capture organizational complexity—political dynamics, informal channels, and tool fragmentation in real environments. LLM outputs exhibit inherent variability; we mitigate this with several controls but it is not eliminated. Specifically, the Analyst runs with adaptive thinking disabled and effort set to low (the prompt is the primary lever, not the reasoning depth), the system prompt is version-controlled and the version hash is stored alongside each classification, the JSON output schema is enforced server-side via structured outputs (so format drift cannot silently corrupt downstream parsing), and the system prompt is identical across batches (allowing prompt caching to read the same bytes and improving both cost and behavioral consistency). What we do not yet measure is run-to-run variance: a single static pass was reported here, and replicate runs against the same dataset to quantify variance are part of the planned full simulation. Finally, reclassification cycles

in very large projects may introduce latency not measured in our single-project design.

The correct comparison is not between this system and perfect risk assessment—it is between this system and current practice. Today, a PM manually identifies 30–40 risks biased by personal experience and rarely updates the register during execution. When a PM misses a risk, the failure is *silent and invisible*; when the AI misclassifies, the error is *traceable, auditable, and correctable*. Moreover, the relevant comparison is not AI versus human, but pattern-structured processing versus the same LLM producing an unstructured flat list—without triage architecture, AI generates the same risks but delivers undifferentiated noise.

6 Conclusions and Next Steps

This paper operationalizes the theoretical framework proposed in [1] into SHARP, defining a semantic triage system, clear human-AI responsibilities at each tier, dynamic reclassification, and a four-layer safety architecture.

Three key design decisions emerged. First, risk classification must be *semantic, not numerical*: LLMs interpret contextual rules in ways probability-impact matrices cannot. Second, the human contribution shifts from *reviewing individual risks* to *designing classification policies*—a reconceptualization of the PM's role in AI-augmented risk management. Third, reliability depends not on eliminating errors but on ensuring the dangerous error (critical risk classified as low-priority) is architecturally minimized.

The framework is initially evaluated through a minimal static classification pass over a synthetic project with 80 planted-ground-truth risks. The results support the feasibility of the triage mechanism and the conservative-escalation logic of the safety architecture, while full multi-event validation with dynamic reclassification, clustering, and weak-signal detection remains future work. Further future work includes empirical validation with practicing project managers, operationalization of the remaining collaboration patterns (clustering, weak-signal detection, cross-team synthesis), and development of organizational maturity models for AI-augmented risk management.

Disclosure of Interests The author has no competing interests to declare that are relevant to the content of this article.

References

1. Bara Iniesta, M., Lostumbo, M.: Human-AI collaboration in high-volume project risk management: A cross-domain framework transfer study. In: International Summer Conference on Intelligent Systems & Decision Making: Human Insights in the Era of AI. University of Catania, Italy (2025)
2. Bara Iniesta, M.: Augmenting human decision-making in risk management: An LLM-based framework for enhancing project risk register quality. In: International

Summer Conference on Intelligent Systems & Decision Making: Human Insights in the Era of AI. University of Catania, Italy (2025)
3. Mohamed, A.A., Mohammed, A., Saud, S., Veedu, A.K., Zubair, S.N., Al Sayari, A.S.: AI-enabled annexure parsing for oil & gas project exhibits. In: Society of Petroleum Engineers—ADIPEC 2024 (2024). https://doi.org/10.2118/222187-MS
4. Sherif, A., Salloum, S.A., Shaalan, K.: Systematic review for knowledge management in Industry 4.0 and ChatGPT applicability as a tool. In: Studies in Big Data (2024). https://doi.org/10.1007/978-3-031-52280-2_19
5. Jalalvand, F., Baruwal Chhetri, M., Nepal, S., Paris, C.: Alert prioritisation in security operations centres: A systematic survey on criteria and methods. ACM Computing Surveys **57**(2) (2024). https://doi.org/10.1145/3695462
6. Huang, Y., Wang, X., Zhang, Y., Chen, L., Zhang, H.: Application of human-in-the-loop hybrid augmented intelligence approach in security inspection system. Frontiers in Artificial Intelligence **8** (2025). https://doi.org/10.3389/frai.2025.1518850
7. Černý, J., Potančok, M., Castro Hernandez, E.: Toward a typology of weak-signal early alert systems: Functional early warning systems in the post-COVID age. Online Information Review **46**(5), 904–919 (2022). https://doi.org/10.1108/OIR-11-2020-0513
8. Al-Hussaini, S., Guan, Y., Gregory, J.M., Pollard, K., Khooshabeh, P., Gupta, S.K.: Assessing the impact of alerts on the human supervisor's decision-making performance in multi-robot missions. ACM Transactions on Human-Robot Interaction **14**(1) (2024). https://doi.org/10.1145/3689828
9. Acebes, F., González-Varona, J.M., López-Paredes, A., Pajares, J.: Beyond probability-impact matrices in project risk management: A quantitative methodology for risk prioritisation. arXiv preprint, arXiv:2405.20679 (2024)
10. Almalki, S.S.: AI-driven decision support systems in agile software project management: Enhancing risk mitigation and resource allocation. Systems **13**(3) (2025). https://doi.org/10.3390/systems13030208
11. Metwally, A.B.M., Ali, S.A.M., Mohamed, A.T.I.: Thinking responsibly about responsible AI in risk management: The darkside of AI in RM. In: 2024 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems (ICETSIS 2024), pp. 1–5 (2024). https://doi.org/10.1109/ICETSIS61505.2024.1045968435